Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Chapter 10

Sparse matrices in MATLAB

Almost all MATLAB operators and functions work seamlessly on both sparse and full matrices.[15] It is possible to write an efficient MATLAB M-file that can operate on either full or sparse matrices with no changes to the code. In MATLAB, "sparse" is an attribute of the data structure used to represent a matrix.

Sparsity propagates in MATLAB; if a function or operator has sparse operands, the result is usually sparse. A fixed set of rules determines the storage class (sparse or full) of the result. In general, unary functions and operators return a result of the same storage class as the input. For example, `chol(A)` is sparse if `A` is sparse and full otherwise. The result of a binary operator (`A+B`, for example) is sparse if both `A` and `B` are sparse and full if both `A` and `B` are full. If the operands are mixed, the result is usually full, unless the operation preserves sparsity (`[A B]`, `[A;B]`, and `A.*B` are sparse if either `A` or `B` are sparse, for example). The submatrix `A(i,j)` has the same type as `A`, unless it is a scalar (in which case `A(i,j)` is full). Submatrix assignment (`A(i,j) = ...`) leaves the storage class of `A` unchanged.

10.1 Creating sparse matrices

There are many ways of creating a sparse matrix in MATLAB, most of which rely on the MATLAB `sparse` function. Usually the fastest method is to create the triplet form first and then use `sparse`, much like how `cs_entry`, `cs_compress`, and `cs_dupl` can be used. Avoid using `for` loops. This example creates an n^2-by-n^2 matrix, corresponding to the second difference operator applied to an n-by-n mesh.

```
function A = mesh2d2 (n)
% create an n-by-n 2D mesh for the 2nd difference operator
nn = 1:n^2 ;
ii = [nn-n ; nn-1 ; nn ; nn+1 ; nn+n] ;
jj = repmat (nn, 5, 1) ;
xx = repmat ([-1 -1 4 -1 -1]', 1, n^2) ;
keep = find (ii >= 1 & ii <= n^2 & jj >= 1 & jj <= n^2) ;
A = sparse (ii (keep), jj (keep), xx (keep)) ;
```

[15] Since MATLAB refers to matrices as either `sparse` or `full`, that nomenclature is used here.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

If `for` loops are unavoidable, at least try to create subsets of more than one entry at a time. This function computes the same matrix as `mesh2d2`. Preallocating a matrix at its final size is better than repeatedly appending new entries to it.

```
function A = mesh2d1 (n)
% create an n-by-n 2D mesh for the 2nd difference operator
ii = zeros (5*n^2, 1) ;     % preallocate ii, jj, and xx
jj = zeros (5*n^2, 1) ;
xx = zeros (5*n^2, 1) ;
k = 1 ;
for j = 0:n-1
    for i = 0:n-1
        s = j*n+i + 1 ;
        ii (k:k+4) = [(j-1)*n+i j*n+(i-1) j*n+i j*n+(i+1) (j+1)*n+i ] + 1 ;
        jj (k:k+4) = [s s s s s] ;
        xx (k:k+4) = [-1 -1 4 -1 -1] ;
        k = k + 5 ;
    end
end
% remove entries beyond the boundary
keep = find (ii >= 1 & ii <= n^2 & jj >= 1 & jj <= n^2) ;
A = sparse (ii (keep), jj (keep), xx (keep)) ;
```

Here is an example that creates a random finite-element matrix.

```
function A = frand (n,nel,s)
% A = frand (n,nel,s) creates an n-by-n sparse matrix consisting of nel finite
% elements, each of which are of size s-by-s with random symmetric nonzero
% pattern, plus the identity matrix.
ss = s^2 ;
nz = nel*ss ;
ii = zeros (nz,1) ;
jj = zeros (nz,1) ;
xx = zeros (nz,1) ;
k = 1 ;
for e = 1:nel
    i = 1 + fix (n * rand (s,1)) ;
    i = repmat (i, 1, s) ;
    j = i' ;
    x = rand (s,s) ;
    ii (k:k+ss-1) = i (:) ;
    jj (k:k+ss-1) = j (:) ;
    xx (k:k+ss-1) = x (:) ;
    k = k + ss ;
end
A = sparse (ii,jj,xx,n,n) + speye (n) ;
```

The `cs_frand` function is the CSparse version of `frand`.

```
cs *cs_frand (int n, int nel, int s)
{
    int ss = s*s, nz = nel*ss, e, i, j, *P ;
    cs *A, *T = cs_spalloc (n, n, nz, 1, 1) ;
    if (!T) return (NULL) ;
    P = cs_malloc (s, sizeof (int)) ;
    if (!P) return (cs_spfree (T)) ;
    for (e = 0 ; e < nel ; e++)
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
    {
        for (i = 0 ; i < s ; i++) P [i] = rand () % n ;
        for (j = 0 ; j < s ; j++)
        {
            for (i = 0 ; i < s ; i++)
            {
                cs_entry (T, P [i], P [j], rand () / (double) RAND_MAX) ;
            }
        }
    }
    for (i = 0 ; i < n ; i++) cs_entry (T, i, i, 1) ;
    A = cs_compress (T) ;
    cs_spfree (T) ;
    return (cs_dupl (A) ? A : cs_spfree (A)) ;
}
```

The *worst* way to create a sparse matrix is with a statement `A(i,j) = ...`, where `i` and `j` are scalars. Below are four methods of creating the same matrix `A`. The first method is an example of what *not* to do.

```
% method 1: A(i,j) = ...
rand ('state', 0) ;
A = sparse (n,n) ;
for k = 1:nz
    % compute some arbitrary entry and add it into the matrix
    i = 1 + fix (n * rand (1)) ;
    j = 1 + fix (n * rand (1)) ;
    x = rand (1) ;
    A (i,j) = A (i,j) + x ; % VERY slow, esp. if A(i,j) not already nonzero!
end
```

A better method, if the matrix is to be constructed one entry at a time, is to preallocate a triplet form, fill it, and then convert it to a sparse form.

```
% method 2: triplet form, one entry at a time
rand ('state', 0) ;
ii = zeros (nz, 1) ;    % preallocate ii, jj, and xx
jj = zeros (nz, 1) ;
xx = zeros (nz, 1) ;
for k = 1:nz
    % compute some arbitrary entry and add it into the matrix
    ii (k) = 1 + fix (n * rand (1)) ;
    jj (k) = 1 + fix (n * rand (1)) ;
    xx (k) = rand (1) ;
end
A = sparse (ii,jj,xx) ;
```

If the number of entries is unknown, the size of the triplet matrix can be increased as needed, just like `cs_entry`.

```
% method 3: triplet form, one entry at a time, pretend nz is unknown
rand ('state', 0) ;
len = 16 ;
ii = zeros (len, 1) ;
jj = zeros (len, 1) ;
xx = zeros (len, 1) ;
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
for k = 1:nz
    % compute some arbitrary entry and add it into the matrix
    if (k > len)
        % double the size of ii,jj,xx
        len = 2*len ;
        ii (len) = 0 ;
        jj (len) = 0 ;
        xx (len) = 0 ;
    end
    ii (k) = 1 + fix (n * rand (1)) ;
    jj (k) = 1 + fix (n * rand (1)) ;
    xx (k) = rand (1) ;
end
A = sparse (ii (1:k), jj (1:k), xx (1:k)) ;
```

Of course, the best method is to avoid `for` loops completely.

```
% method 4: avoid the for loop
rand ('state', 0) ;
e = rand (3, nz) ;
e (1,:) = 1 + fix (n * e (1,:)) ;
e (2,:) = 1 + fix (n * e (2,:)) ;
A = sparse (e (1,:), e (2,:), e (3,:)) ;
```

Each of the above four methods constructs the same matrix `A`. The first is exceedingly slow, taking $O(|A|^2)$ time. Methods 2 and 3 take about the same time, but method 4 is much faster (methods 2, 3, and 4 all take $O(|A|)$ time, however).

Finally, never create a full matrix `A` and then convert it to sparse. For example, `A=sparse(eye(n))` takes $O(n^2)$ time and memory, but `A=speye(n)` takes only $O(n)$ time and memory. The difference is dramatic for large `n`.

10.2 Sparse matrix functions and operators

The following is a list of the primary functions and operators for sparse matrices in MATLAB 7.2.

referencing and assigning submatrices:	
`subsref`	`...=A(i,j)` extracts a sparse submatrix of `A`. Matrix permutation is an important special case. `C=A(p,q)` is the same as $C = PAQ$ if `p` and `q` are permutations of `1:m` and `1:n`, respectively, that represent the permutation matrices P and Q (where `[m n]=size(A)`).
`subasgn`	`A(i,j)=...` modifies a sparse submatrix of `A`.
`subsindex`	`...=A(k)`, one-dimensional indexing.
elementary sparse matrices:	
`spdiags`	A generalization of `diag` that constructs a matrix from diagonals or extracts diagonals of a matrix.
`speye`	`I=speye(n)` is the sparse identity matrix; `I=speye(m,n)` is `m`-by-`n`.
`sprand`	`A=sprand(m,n,d)` is a random `m`-by-`n` sparse matrix with about `d*m*n` nonzero entries with values uniformly distributed in the range $[0, 1]$. `A=sprand(S)` is a random matrix with the same pattern as `S`.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

elementary sparse matrices, continued:	
`sprandn`	Identical to `sprand` but with normally distributed random entries.
`sprandsym`	`A=sprandsym(n,d)` is a symmetric random matrix with about dn^2 entries. `A=sprandsym(n,d,rc,kind)` is also positive definite with a reciprocal condition number of `rc` (`kind` selects the method).

full to sparse conversion:	
`find`	`[i j x]=find(A)` extracts the triplet form of a matrix `A`; it is the opposite of `sparse`. With one output, `i=find(A)` returns the one-dimensional indices `i` of the nonzero entries in `A`.
`full`	`C=full(A)` converts a sparse matrix `A` into a full matrix `C`.
`sparse`	`A=sparse(C)` converts a full matrix `C` into sparse matrix `A`. `A=sparse(i,j,x,m,n,s)` converts a triplet form into an `m`-by-`n` sparse matrix `A` with space for `s` entries. `A=sparse(i,j,x,m,n)` uses `s=length(x)`. `A=sparse(i,j,x)` uses `m=max(i)` and `n=max(j)`, just like `cs_sparse`. `A=sparse(m,n)` is a sparse `m`-by-`n` matrix with all zero entries.
`spconvert`	`A=spconvert(x)` is `A=sparse(x(:,1),x(:,2),x(:,3))` if `x` has 3 columns. `A` is complex if `size(x,2)=4`.

working with sparse matrices:	
`issparse`	`issparse(A)` is `1` if `A` is sparse and `0` otherwise.
`nnz`	`nnz(A)` is the number of nonzeros in `A` (`A->p[A->n]` in CSparse).
`nonzeros`	`x` from `[i,j,x]=find(A)` (`A->x` in CSparse).
`nzmax`	`nzmax(A)` is the maximum number of nonzeros `A` can hold; this is increased when necessary (`A->nzmax` in CSparse).
`spalloc`	`A=spalloc(m,n,nzmax)` allocates a zero `m`-by-`n` sparse matrix with space for `nzmax` entries.
`spfun`	Applies a function to the entries of a matrix. `C=spfun(@f,A)` is a matrix with the same nonzero pattern as `A`, where `C(i,j)=f(A(i,j))` (zero entries are removed from `C`).
`spones`	`C=spones(A)` is a binary sparse matrix with the pattern of `A`.
`spparms`	Parameters for sparse methods. Try `spparms('spumoni',2)`.
`spy`	`spy(A)` plots a picture of a sparse matrix.

ordering methods:	
`amd`	Approximate minimum degree algorithm used in backslash, `lu`, and `chol`. `p=amd(A)` orders `A+A'` so that `chol(A(p,p))` tends to be sparser than `chol(A)`.
`colamd`	Column approximate minimum degree ordering used in backslash and `lu`. `q=colamd(A)` finds a column ordering so that `lu(A(:,q))` tends to be sparser than `lu(A)`. Dense rows are removed from `A` prior to ordering `A` (like `cs_amd(A,2)`). For QR factorization, use `q=colamd(A,[n m])`, where `[m n]=size(A)`, so that dense rows in `A` are not ignored (like `cs_amd(A,3)`). This finds a column ordering `q` so that `qr(A(:,q))` tends to be sparser than `qr(A)`.
`colperm`	Sorts columns by increasing degree.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

ordering methods, continued:	
`dmperm`	`[p,q,r,s]=dmperm(A)` is the Dulmage–Mendelsohn decomposition of `A`, where `C=A(p,q)` is in block upper triangular form. The `k`th block is `C(r(k):r(k+1)-1,s(k):s(k+1)-1)`.
`randperm`	`p=randperm(n)` is a random permutation of `1:n`.
`symamd`	Approximate minimum degree, based on `colamd`; `p=symamd(A)`, is like `p=amd(A)`, just slower. `symamd` orders the matrix with pattern `tril(A)+tril(A)'`.
`symrcm`	`p=symrcm(A)` finds a reverse Cuthill–McKee ordering so that the entries of `C(p,p)` tend to be close to the diagonal, where `C=A+A'`.
linear algebra:	
`cholinc`	Incomplete Cholesky factorization. See Problem 4.13.
`condest`	`c=condest(A)` is a lower bound for `cond(full(A),1)` that is much less expensive to compute.
`eigs`	`e=eigs(A)` returns the six largest eigenvalues of a square matrix `A`, using a method that is less expensive than `e=eig(A)`. `[V,e]=eigs(A)` also returns the eigenvectors `V`. Many more options are available. Based on ARPACK.
`luinc`	Incomplete LU factorization. See Problem 6.13.
`normest`	`s=normest(A)` is an estimate of the 2-norm `norm(full(A))` that is much less expensive to compute.
`spaugment`	`S=spaugment(A,c)` is the augmented system `S=[c*I A;A' 0]`.
`sprank`	`s=sprank(A)` is the structural rank of `A` (the size of the maximum matching).
`svds`	`s=svds(A)` finds the six largest singular values of `A`. Many more options are available.
sparse factorization methods:	
`lu`	LU factorization. `[L,U,P]=lu(A)` returns `L*U=P*A`, using the left-looking GPLU algorithm (much like `cs_lu`) with no fill-reducing ordering. `[L2,U]=lu(A)` is the same, except `L2=P'*L` is returned. `[L,U,P,Q]=lu(A)` uses UMFPACK, where `Q` is a fill-reducing ordering and `P` is for both threshold partial pivoting and reducing fill-in. A second input to `lu` controls the threshold.
`qr`	Givens-rotation-based QR factorization. `R=qr(A)` computes just `R` and not `Q`; this uses much less memory than `[Q R]=qr(A)`. `R=qr(A,0)` returns `R` with just `min(size(A))` rows. `[x,R]=qr(A,b)` solves the least squares problem to minimize $\|Ax - b\|_2$. No column ordering is used to reduce `nnz(R)`; use `q=colamd(A,[n m])` and `qr(A(:,q))` to reduce fill-in.
`chol`	Cholesky factorization of `A`, using CHOLMOD. `R=chol(A)` returns an upper triangular matrix where `R'*R=A`. No fill-reducing ordering is used; use `p=amd(A)` or `p=symamd(A)` and `chol(A(p,p))` to reduce fill-in.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

operators (highlights only; all MATLAB operators work for sparse matrices):	
`\`	Backslash, or `mldivide`. See Section 8.5.
`/`	Slash, or `mrdivide`. See Section 8.5.
`'`	`C=A'` is the transpose of `A` (complex conjugate transpose if `A` is complex).
`+`	`C=A+B` adds two matrices. `C` is sparse if `A` and `B` are sparse.
`-`	`C=A-B` subtracts two matrices. `C` is sparse if `A` and `B` are sparse.
`*`	`C=A*B` multiplies two matrices. `C` is sparse if `A` and `B` are sparse.
`;`	`C=[A;B]` concatenates `A` and `B` vertically; `A` and `B` must have the same number of columns. `C` is sparse if `A` or `B` are sparse.
`,`	`C=[A,B]` concatenates `A` and `B` horizontally; `A` and `B` must have the same number of rows. `C` is sparse if `A` or `B` are sparse.

iterative methods:

Iterative methods exploit sparsity when solving $Ax = b$ by not factorizing A. They typically rely on repeated matrix-vector multiplications. Methods in MATLAB include `bicg`, `bicgstab`, `cgs`, `gmres`, `lsqr`, `minres`, `pcg`, `qmr`, and `symmlq`.

tree and graph operations:	
`etree`	`parent=etree(A)` is the elimination tree of `triu(A)+triu(A)'`. `[parent post]=etree(A)` also returns the elimination tree postordering. `etree(A,'col')` finds the elimination tree of `A'*A`.
`etreeplot`	`etreeplot(A)` plots a picture of the elimination tree of `A+A'`.
`gplot`	`gplot(A,xy)` plots a picture of the undirected graph of `A+A'`, where the `n`-by-2 matrix `xy` gives the x-y coordinates of each node.
`symbfact`	Symbolic Cholesky factorization. `c=symbfact(A)` is a vector of column counts of the Cholesky factor `L=chol(A)'`, where only `triu(A)` is accessed. `symbfact(A,'col')` analyzes `A'*A` but does not form it. Additional outputs are `[c h parent post R]=symbfact(...)`, where `h` is the height of the elimination tree, `parent` is the tree, `post` is the postordering of the tree, and `R` is a binary matrix with the same pattern `chol(A)`.
`treelayout`	`[x,y,h]=treelayout(parent)` finds x-y coordinates for the nodes of a tree, and the height `h` of the tree, for use in `treeplot`.
`treeplot`	`treeplot(parent)` plots a picture of the elimination tree.

functions that partially work on sparse matrices or that have sparse substitutes:	
`cholupdate`	Rank-1 update/downdate of full Cholesky factorization. Use CHOLMOD or `cs_updown` for the sparse case.
`cond`	Use `condest` instead.
`eig`	Use `eigs` instead or `d=eig(A)` for sparse symmetric `A`.
`norm`	Works for 1-norm, ∞-norm, vector 2-norm, and Frobenius norm. Use `normest(A)` to estimate the 2-norm of a sparse matrix `A`.
`poly`	Only works if `A` is symmetric.
`svd`	Use `svds` instead.

functions and features that do not work on sparse matrices:

Functions and features of MATLAB that do not work at all for sparse matri-

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

ces include N-dimensional arrays for $N > 2$, different types (only double and complex double are available), `airy`, `bessel`, `besselj`, `bessely`, `besseli`, `besselk`, `besselh`, `betainc`, `bitand`, `bitcmp`, `bitor`, `bitxor`, `bitset`, `bitget`, `bitshift`, `complex`, `condeig`, `conv2`, `convn`, `deconv`, `erf`, `erfc`, `erfcx`, `fft`, `fft2`, `fftn`, `filter`, `filter2`, `funm`, `gamma`, `gammaln`, `gsvd`, `hess`, `histc`, `ifft`, `ifftn`, `linsolve`, `logm`, `lsqnonneg`, `null`, `ordeig`, `ordqz`, `ordschur`, `orth`, `qz`, `pinv`, `psi`, `rank`, `rcond`, `reallog`, `realpow`, `realsqrt`, `residue`, `rsf2csf`, `schur`, `sqrtm`, `ss2zp`, `subspace`, `surfnorm`, and `tzero`.

The following accept sparse inputs but produce full outputs: `ellipj`, `ellipke`, `erfcinv`, `erfinv`, `expint`, `gammainc`, `legendre`, `polyeig`, `polyval`, and `polyvalm`.

10.3 CSparse MATLAB interface

The MATLAB mexFunction interface for CSparse allows all of CSparse to be used within MATLAB. MATLAB already includes most of the functionality of CSparse (and much more) with the exception of the rank-1 sparse Cholesky update/downdate. Most CSparse functions are about as fast, or faster, than built-in functions in MATLAB 7.2 with notable exceptions of `chol` and `lu` because they rely on dense matrix kernels. See `CSparse/MATLAB/cs_install.m` to compile and install CSparse for use in MATLAB.

`cs_add`: sparse matrix addition

Usage: `C = cs_add(A,B,alpha,beta)`
MATLAB equivalent: `C = alpha*A+beta*B`
Adds two sparse matrices. `alpha` and `beta` default to 1 if not present.
See also `cs_multiply`, `cs_gaxpy`, `plus`, `minus`.

`cs_amd`: approximate minimum degree ordering

Usage: `p = cs_amd(A,order)`
MATLAB equivalent: `p = amd(A)`, `p = colamd(A)`, or `p = symamd(A)`
Approximate minimum degree ordering. The `order` parameter is optional. The default is `order=1`, which orders `A+A'`. `order=2` orders `S'*S`, where `S = A` except that dense rows are removed from `S`. `order=3` orders `A'*A`.
See also `amd`, `colamd`, `symamd`.

`cs_chol`: sparse Cholesky factorization

Usage: `[L,p] = cs_chol(A,drop)`
MATLAB equivalent: `p=amd(A) ; L=chol(A(p,p))'`, or `L=chol(A')'`
Factorizes `A` or `A(p,p)` into its Cholesky factorization `L*L'`. `drop` is optional; if zero, numerically zero entries are not dropped from `L`. The default is to drop these entries (`drop = 1`). With one output, no fill-reducing ordering is used.
See also `cs_amd`, `cs_updown`, `chol`, `amd`, `symamd`.

`cs_cholsol`: solve $Ax = b$ using a sparse Cholesky factorization

Usage: `x = cs_cholsol(A,b,order)`

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

MATLAB equivalent: `x = A\b`

Solves $Ax = b$ using a sparse Cholesky factorization. `b` must be a full vector. If `order` is present, `cs_amd(A,order)` is used.

See also `cs_chol`, `cs_amd`, `cs_lusol`, `cs_qrsol`, `mldivide`.

`cs_counts`: column counts for sparse Cholesky factorization

Usage: `c = cs_counts(A,mode)`

MATLAB equivalent: `c = symbfact(A)` or `c = symbfact(A,'col')`

Returns a vector of the column counts of `L`, `c = sum(spones(chol(A)'))`, except that it is computed more efficiently. `c = cs_counts(A,'col')` computes the counts for the factorization of `A'*A`.

See also `symbfact`.

`cs_dmperm`: maximum matching or Dulmage–Mendelsohn decomposition

Usage: `[p,q,r,s,cc,rr] = cs_dmperm(A)` or `p = cs_dmperm(A)`

MATLAB equivalent: `[p,q,r,s] = dmperm(A)` or `p = dmperm(A)`

`p = cs_dmperm(A)` finds a maximum matching `p` such that `p(j) = i` if column `j` is matched to row `i` or `-1` if column `j` is unmatched. If `A` is square and full structural rank, `p` is a row permutation and `A(p,:)` has a zero-free diagonal. The structural rank of `A` is `sprank(A) = sum(p>0)`.

`[p,q,r,s,cc,rr] = cs_dmperm(A)` finds the Dulmage–Mendelsohn decomposition of `A`. `p` and `q` are permutation vectors. `cc` and `rr` are vectors of length 5. `C = A(p,q)` is split into a 4-by-4 set of coarse blocks

$$C = \begin{bmatrix} A_{11} & A_{12} & A_{13} & A_{14} \\ & & A_{23} & A_{24} \\ & & & A_{34} \\ & & & A_{44} \end{bmatrix},$$

where A_{12}, A_{23}, and A_{34} are square with zero-free diagonals. The columns of A_{11} are the unmatched columns, and the rows of A_{44} are the unmatched rows. Any of these blocks can be empty. In the coarse decomposition, the (i,j)th block is `C(rr(i):rr(i+1)-1,cc(j):cc(j+1)-1)`. In terms of a linear system, `[A11 A12]` is the undetermined part of the system (it is always rectangular and with more columns than rows or 0-by-0), A_{23} is the well-determined part of the system (it is always square), and `[A34 ; A44]` is the overdetermined part of the system (it is always rectangular with more rows than columns or 0-by-0). The structural rank of `A` is `rr(4)-1`. The A_{23} submatrix is further subdivided into block upper triangular form via the fine decomposition (the strongly connected components of A_{23}).

`C(r(i):r(i+1)-1,s(j):s(j+1)-1)` is the (i,j)th block of the fine decomposition. The $(1,1)$ block is the rectangular block `[A11 A12]`, unless this block is 0-by-0. The (b,b) block is the rectangular block `[A34 ; A44]`, unless this block is 0-by-0, where `b = length(r)-1`. All other diagonal blocks are submatrices of A_{23} and are square with a zero-free diagonal.

A second argument provides a seed for a randomized maximum matching.

See also `cs_dmspy`, `cs_dmsol`, `dmperm`, `sprank`, `cs_randperm`.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

cs_dmsol: solve $Ax = b$ using a Dulmage–Mendelsohn decomposition

See Section 8.4 for a complete description of the `cs_dmsol` M-file.

cs_dmspy: plot a Dulmage–Mendelsohn decomposition

Usage: `[p,q,r,s,cc,rr] = cs_dmspy(A,res)`

MATLAB equivalent: `[p,q,r,s] = dmperm(A) ; spy(A(p,q))`

Plots a picture of the Dulmage–Mendelsohn decomposition of a matrix. It first computes `[p,q,r,s,cc,rr] = cs_dmperm(A)`, does `cspy(A(p,q))`, and then draws boxes around the coarse and fine decompositions. A second input argument (`cs_dmspy(A,res)`) changes the resolution of the image to `res`-by-`res` (default resolution is 256). If `res` is zero, `spy` is used instead of `cspy`.

See also `cs_dmperm`, `cs_dmsol`, `dmperm`, `sprank`, `spy`, `cspy`.

cs_droptol: remove small entries from a sparse matrix

Usage: `C = cs_droptol(A,tol)`

MATLAB equivalent: `A = A.*(abs(A)>tol)`

Removes small entries from a sparse matrix (those with magnitude $\leq$ `tol`).

cs_esep: find an edge separator

See Section 7.6 for a complete description of the `cs_esep` M-file.

cs_etree: elimination tree of `A` or `A'*A`

Usage: `[parent,post] = cs_etree(A,mode)`

MATLAB equivalent: `[parent,post] = etree(A)`

`parent = cs_etree(A)` returns the elimination tree of `A`. With a second input, `parent = cs_etree(A,'col')` returns the elimination tree of `A'*A`. For the symmetric case (`cs_etree(A)`), only `triu(A)` is used.

`[parent,post] = cs_etree(...)` also returns a postorder of the tree.

See also `etree`, `treeplot`.

cs_gaxpy: sparse matrix times vector

Usage: `z = cs_gaxpy(A,x,y)`

MATLAB equivalent: `z = A*x+y`

`x` and `y` must be full vectors.

See also `plus`, `mtimes`.

cs_lsolve: solve a sparse lower triangular system, $Lx = b$

Usage: `x = cs_lsolve(L,b)`

MATLAB equivalent: `x = L\b`

Solves a sparse lower triangular system. `L` must be lower triangular with a zero-free diagonal. `b` can be a full or sparse vector.

See also `cs_ltsolve`, `cs_usolve`, `cs_utsolve`, `mldivide`.

cs_ltsolve: solve a sparse upper triangular system, $L^T x = b$

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Usage: `x = cs_ltsolve(L,b)`

MATLAB equivalent: `x = L'\b`

Solves a sparse upper triangular system. `L` must be lower triangular with a zero-free diagonal. `b` must be a full vector.

See also `cs_lsolve`, `cs_usolve`, `cs_utsolve`, `mldivide`.

`cs_lu`: sparse LU factorization

Usage: `[L,U,p,q] = cs_lu(A,tol)`

MATLAB equivalent: `[L,U,P,Q] = lu(A,tol)`

`[L,U,p] = cs_lu(A)` factorizes `A(p,:)` into `L*U`.

`[L,U,p] = cs_lu(A,tol)` factorizes `A(p,:)` into `L*U`. Entries on the diagonal are given preference in partial pivoting.

`[L,U,p,q] = cs_lu(A)` factorizes `A(p,q)` into `L*U`, using a fill-reducing ordering `q = cs_amd(A,2)`. Normal partial pivoting is used.

`[L,U,p,q] = cs_lu(A,tol)` factorizes `A(p,q)` into `L*U`, using a fill-reducing ordering `q = cs_amd(A)`. Entries on the diagonal are given preference in partial pivoting. With a pivot tolerance `tol`, the entries in `L` have magnitude `1/tol` or less. `tol = 1` is normal partial pivoting (but with the `q = cs_amd(A)` ordering). `tol = 0` ensures `p = q`. `0<tol<1` is relaxed partial pivoting; the diagonal is selected if it is at least `tol*max(abs(A(:,k)))`.

See also `cs_amd`, `lu`, `umfpack`, `amd`, `colamd`.

`cs_lusol`: solve $Ax = b$ using LU factorization

Usage: `x = cs_lusol(A,b,order,tol)`

MATLAB equivalent: `x = A\b`

`x = cs_lusol(A,b)` computes `x=A\b`, where A is sparse and square, and `b` is a full vector. The ordering `cs_amd(A,2)` is used. `x = cs_lusol(A,b,1)` also computes `x=A\b` but uses the `cs_amd(A)` ordering with diagonal preference (default `tol=0.001`). If `order` is present, `cs_amd(A,order)` is used.

See also `cs_lu`, `cs_amd`, `cs_cholsol`, `cs_qrsol`, `mldivide`.

`cs_make`: compiles CSparse for use in MATLAB

Usage: `cs_make`

See also `mex`. Type `help cs_make` in MATLAB for more information, including instructions on how to add new mexFunctions to CSparse.

`cs_multiply`: sparse matrix multiply

Usage: `C = cs_multiply(A,B)`

MATLAB equivalent: `C = A*B`

See also `cs_gaxpy`, `cs_add`, `mtimes`.

`cs_nd`: generalized nested dissection

See Section 7.6 for a complete description of the `cs_nd` M-file.

`cs_nsep`: find a node separator

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

See Section 7.6 for a complete description of the `cs_nsep` M-file.

`cs_permute`: permute a sparse matrix

Usage: `C = cs_permute(A,p,q)`
MATLAB `C = A(p,q)`
See also `cs_symperm`, `subsref`.

`cs_print`: print a sparse matrix

Usage: `cs_print(A,brief)`
MATLAB equivalent: `A` with no semicolon
`cs_print(A)` prints a sparse matrix. `cs_print(A,1)` prints just a few entries.

`cs_qleft`: apply Householder vectors on the left

See Section 5.3 for a description of the `cs_qleft` M-file.

`cs_qr`: sparse QR factorization

Usage: `[V,beta,p,R,q] = cs_qr(A)`
MATLAB equivalent: `q=colamd(A,[n m]) ; [Q,R] = qr(A(:,q))`
`[V,beta,p,R] = cs_qr(A)` computes the QR factorization of `A(p,:)`.
`[V,beta,p,R,q] = cs_qr(A)` computes the QR factorization of `A(p,q)`. The fill-reducing ordering `q` is found via `q = cs_amd(A,3)`.
`A` must be `m`-by-`n` with $\texttt{m} \geq \texttt{n}$. If `A` is structurally rank deficient, additional empty rows may have been added to `V` and `R`. The orthogonal factor `Q` can be obtained via `Q = cs_qright(V,Beta,p,speye(size(V,1)))`.
See also `cs_amd`, `cs_qr`, `cs_qright`, `cs_dmperm`, `qr`, `colamd`.

`cs_qright`: apply Householder vectors on the right

See Section 5.3 for a description of the `cs_qright` M-file.

`cs_qrsol`: solve a sparse least squares problem

Usage: `x = cs_qrsol(A,b,order)`
MATLAB `x = A\b`
`x = cs_qrsol(A,b)` solves the overdetermined least squares problem to find `x` that minimizes `norm(A*x-b)`, where `b` is a full vector. `A` is `m`-by-`n` with $\texttt{m} \geq \texttt{n}$. If `order` is present, `cs_amd(A,order)` is used. If `A` has fewer rows than columns, an underdetermined problem is solved.
See also `cs_qr`, `cs_amd`, `cs_lusol`, `cs_cholsol`, `mldivide`.

`cs_randperm`: random permutation

Usage: `p = cs_randperm(n,seed)`
MATLAB equivalent: `p = randperm(n)`

`cs_scc`: strongly connected components of a square sparse matrix

Usage: `[p,r] = cs_scc(A)`

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

MATLAB equivalent: `[p,q,r,s] = dmperm(A+speye(size(A,1)))`

`[p,r] = cs_scc(A)` finds a permutation `p` so that `A(p,p)` is permuted into block upper triangular form. The diagonal of `A` is ignored.

See also `cs_dmperm`, `dmperm`.

`cs_sep`: convert an edge separator into a node separator

See Section 7.6 for a complete description of the `cs_sep` M-file.

`cs_sparse`: convert a triplet form into a MATLAB sparse matrix

Usage: `A = cs_sparse(i,j,x)`

MATLAB equivalent: `A = sparse(i,j,x)`

`A = cs_sparse(i,j,x)` is identical to `A = sparse(i,j,x)`, except that x must be real, and the lengths of `i`, `j`, and `x` must be the same. The MATLAB `sparse` function has many more features than `cs_sparse`.

See also `sparse`, `spconvert`.

`cs_sqr`: symbolic QR or LU ordering and analysis

Usage: `[vnz,rnz,parent,c,leftmost,p,q] = cs_sqr(A)`

MATLAB equivalent: `[c,parent] = symbfact (A, 'col')`

`[vnz,rnz,parent,c,leftmost,p] = cs_sqr(A)` computes the symbolic QR factorization of `A(p,:)`. `[vnz,rnz,parent,c,leftmost,p,q] = cs_sqr(A)` computes the symbolic QR factorization of `A(p,q)`. The fill-reducing ordering `q` is found via `q=cs_amd(A,3)`. `vnz` is the number of entries in the matrix of Householder vectors, `V`. `rnz` is the number of entries in `R`. `parent` is the elimination tree. `c(i)` is the number of entries in `R(i,:)`. `leftmost(i)` is `min(find(A(i,q)))`. `p` is the row permutation used to ensure `R` has a symbolically zero-free diagonal. `q` is the fill-reducing ordering if requested.

See also `cs_amd`, `cs_qr`.

`cs_symperm`: symmetric permutation of a symmetric matrix

Usage: `C = cs_symperm(A,p)`

MATLAB equivalent: `C = triu(A)+triu(A,1)' ; C = triu(C(p,p))`

`C = cs_symperm(A,p)` computes `C = A(p,p)` but accesses only the upper triangular part of `A` and returns `C` upper triangular (`A` and `C` are symmetric with just their upper triangular parts stored). `A` must be square.

See also `cs_permute`, `subsref`, `triu`.

`cs_transpose`: transpose a sparse matrix

Usage: `C = cs_transpose(A)`

MATLAB equivalent: `C = A'`

See also `transpose`, `ctranspose`.

`cs_updown`: rank-1 sparse Cholesky update/downdate

Usage: `L = cs_updown(L,c,parent,sigma)`

MATLAB equivalent: `L = sparse(choldupate(full(L),c,sigma))`

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

`L=cs_updown(L,c,parent)` computes the rank-1 update `L=chol(L*L'+c*c')'`, where `parent` is the elimination tree of `L`. `c` must be a sparse column vector, and `find(c)` must be a subset of `find(L(:,k))`, where `k=min(find(c))`.

`L=cs_updown(L,c,parent,'-')` is the downdate `L=chol(L*L'-c*c')'`.

`L=cs_updown(L,c,parent,'+')` is the update `L=chol(L*L'+c*c')'`.

Updating/downdating is much faster than refactorization with `cs_chol` or `chol`. `L` must not have an entries dropped due to numerical cancellation.

See also `cs_etree`, `cs_chol`, `etree`, `cholupdate`, `chol`.

`cs_usolve`: solve a sparse upper triangular system $Ux = b$

Usage: `x = cs_usolve(U,b)`

MATLAB equivalent: `x = U\b`

Solves a sparse upper triangular system. `U` must be upper triangular with a zero-free diagonal. `b` can be a full or sparse vector.

See also `cs_lsolve`, `cs_ltsolve`, `cs_utsolve`, `mldivide`.

`cs_utsolve`: solve a sparse lower triangular system $U'x = b$

Usage: `x = cs_utsolve(U,b)`

MATLAB equivalent: `x = U'\b`

Solves a sparse lower triangular system. `U` must be upper triangular with a zero-free diagonal. `b` must be a full vector.

See also `cs_lsolve`, `cs_ltsolve`, `cs_usolve`, `mldivide`.

`cspy`: plot a sparse matrix in color

Usage: `cspy(A,res)`

`cspy(A)` plots a sparse matrix, in color, with a default resolution of 256-by-256. `cspy(A,res)` changes the resolution to `res`. Entries with tiny absolute value are light tan. Entries with large magnitude are black. Entries in the midrange (the median of the $\log_{10}$ of the nonzero values, $\pm$ one standard deviation) range from light green to deep blue. With no inputs, the color legend of `cspy` is plotted. `[s,M,H] = cspy(A)` returns the scale factor `s`, the image `M`, and colormap `H`.

See also `cs_dmspy`, `spy`.

10.4 Examples

The `cs_demo1`, `cs_demo2`, and `cs_demo3` M-files in the `MATLAB/Demo` directory are MATLAB equivalents of the C demo programs of the same name. They access CSparse via a set of mexFunctions. These demos also plot their results with `cspy`.

A mexFunction interfaces a C or Fortran program to MATLAB. Once compiled, it acts just like an M-file. Its name is always "`mexFunction`" and it always has the same parameters. C- and Fortran-callable MATLAB functions with the prefix `mx` provide access to MATLAB data structures, while functions with `mex` prefixes operate in the MATLAB environment. Below is a sample mexFunction in CSparse that interfaces the `cs_chol` function to MATLAB (the file `cs_chol_mex.c`). It calls `cs_mex_get_sparse` to convert a MATLAB matrix `A` into a CSparse matrix `A`. The

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

matrix is analyzed and factorized with `cs_schol` and `cs_chol`. The `drop` parameter determines whether or not numerically zero entries should be dropped from the matrix (they must be kept for `cs_updown` to work properly). `cs_mex_put_sparse` returns `L` to the MATLAB caller. If two output parameters have been provided, then the permutation `p` is computed and returned to MATLAB via `cs_mex_put_int`.

```
#include "cs_mex.h"
/* cs_chol: sparse Cholesky factorization */
void mexFunction (int nargout, mxArray *pargout [ ], int nargin,
    const mxArray *pargin [ ])
{
    cs Amatrix, *A ;
    int order, n, drop, *p ;
    css *S ;
    csn *N ;
    if (nargout > 2 || nargin < 1 || nargin > 2)
        mexErrMsgTxt ("Usage: [L,p] = cs_chol(A,drop)") ;
    A = cs_mex_get_sparse (&Amatrix, 1, 1, pargin [0]) ;        /* get A */
    n = A->n ;
    order = (nargout > 1) ? 1 : 0 ;                 /* determine ordering */
    S = cs_schol (order, A) ;                       /* symbolic Cholesky */
    N = cs_chol (A, S) ;                            /* numeric Cholesky */
    if (!N) mexErrMsgTxt ("cs_chol failed: not positive definite\n") ;
    drop = (nargin == 1) ? 1 : mxGetScalar (pargin [1]) ;
    if (drop) cs_dropzeros (N->L) ;                 /* drop zeros if requested*/
    pargout [0] = cs_mex_put_sparse (&(N->L)) ;     /* return L */
    if (nargout > 1)
    {
        p = cs_pinv (S->pinv, n) ;                  /* p=pinv' */
        pargout [1] = cs_mex_put_int (p, n, 1, 1) ; /* return p */
    }
    cs_nfree (N) ;
    cs_sfree (S) ;
}
```

`nargin` and `nargout` are the number of input and output parameters. `pargout` and `pargin` are arrays of pointers to the input and output parameters. The `cs_chol` mexFunction makes use of a set of utility routines called `cs_mex_*` shared by all CSparse mexFunctions, in the `cs_mex.c` file, listed below.

```
#include "cs_mex.h"
/* check MATLAB input argument */
void cs_mex_check (int nel, int m, int n, int square, int sparse, int values,
    const mxArray *A)
{
    int nnel, mm = mxGetM (A), nn = mxGetN (A) ;
    if (values)
    {
        if (mxIsComplex (A)) mexErrMsgTxt ("matrix must be real") ;
        if (!mxIsDouble (A)) mexErrMsgTxt ("matrix must be double") ;
    }
    if (sparse && !mxIsSparse (A)) mexErrMsgTxt ("matrix must be sparse") ;
    if (!sparse && mxIsSparse (A)) mexErrMsgTxt ("matrix must be full") ;
    if (nel)
    {
        /* check number of elements */
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
        nnel = mxGetNumberOfElements (A) ;
        if (m >= 0 && n >= 0 && m*n != nnel) mexErrMsgTxt ("wrong length") ;
    }
    else
    {
        /* check row and/or column dimensions */
        if (m >= 0 && m != mm) mexErrMsgTxt ("wrong dimension") ;
        if (n >= 0 && n != nn) mexErrMsgTxt ("wrong dimension") ;
    }
    if (square && mm != nn) mexErrMsgTxt ("matrix must be square") ;
}

/* get a MATLAB sparse matrix and convert to cs */
cs *cs_mex_get_sparse (cs *A, int square, int values, const mxArray *Amatlab)
{
    cs_mex_check (0, -1, -1, square, 1, values, Amatlab) ;
    A->m = mxGetM (Amatlab) ;
    A->n = mxGetN (Amatlab) ;
    A->p = mxGetJc (Amatlab) ;
    A->i = mxGetIr (Amatlab) ;
    A->x = values ? mxGetPr (Amatlab) : NULL ;
    A->nzmax = mxGetNzmax (Amatlab) ;
    A->nz = -1 ;    /* denotes a compressed-col matrix, instead of triplet */
    return (A) ;
}

/* return a sparse matrix to MATLAB */
mxArray *cs_mex_put_sparse (cs **Ahandle)
{
    cs *A ;
    mxArray *Amatlab ;
    A = *Ahandle ;
    Amatlab = mxCreateSparse (0, 0, 0, mxREAL) ;
    mxSetM (Amatlab, A->m) ;
    mxSetN (Amatlab, A->n) ;
    mxSetNzmax (Amatlab, A->nzmax) ;
    cs_free (mxGetJc (Amatlab)) ;
    cs_free (mxGetIr (Amatlab)) ;
    cs_free (mxGetPr (Amatlab)) ;
    mxSetJc (Amatlab, A->p) ;           /* assign A->p pointer to MATLAB A */
    mxSetIr (Amatlab, A->i) ;
    mxSetPr (Amatlab, A->x) ;
    mexMakeMemoryPersistent (A->p) ;    /* ensure MATLAB does not free A->p */
    mexMakeMemoryPersistent (A->i) ;
    mexMakeMemoryPersistent (A->x) ;
    cs_free (A) ;                       /* frees A struct only, not A->p, etc */
    *Ahandle = NULL ;
    return (Amatlab) ;
}

/* get a MATLAB dense column vector */
double *cs_mex_get_double (int n, const mxArray *X)
{
    cs_mex_check (0, n, 1, 0, 0, 1, X) ;
    return (mxGetPr (X)) ;
}
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
/* return a double vector to MATLAB */
double *cs_mex_put_double (int n, const double *b, mxArray **X)
{
    double *x ;
    int k ;
    *X = mxCreateDoubleMatrix (n, 1, mxREAL) ;      /* create x */
    x = mxGetPr (*X) ;
    for (k = 0 ; k < n ; k++) x [k] = b [k] ;       /* copy x = b */
    return (x) ;
}

/* get a MATLAB flint array and convert to int */
int *cs_mex_get_int (int n, const mxArray *Imatlab, int *imax, int lo)
{
    double *p ;
    int i, k, *I = cs_malloc (n, sizeof (int)) ;
    cs_mex_check (1, n, 1, 0, 0, 1, Imatlab) ;
    p = mxGetPr (Imatlab) ;
    *imax = 0 ;
    for (k = 0 ; k < n ; k++)
    {
        i = p [k] ;
        I [k] = i - 1 ;
        if (i < lo) mexErrMsgTxt ("index out of bounds") ;
        *imax = CS_MAX (*imax, i) ;
    }
    return (I) ;
}

/* return an int array to MATLAB as a flint row vector */
mxArray *cs_mex_put_int (int *p, int n, int offset, int do_free)
{
    mxArray *X = mxCreateDoubleMatrix (1, n, mxREAL) ;
    double *x = mxGetPr (X) ;
    int k ;
    for (k = 0 ; k < n ; k++) x [k] = (p ? p [k] : k) + offset ;
    if (do_free) cs_free (p) ;
    return (X) ;
}
```

The `cs_chol.m` M-file provides documentation. Typing `help cs_chol` in the MATLAB Command Window prints the comments in the first part of `cs_chol.m`.

```
function [L,p] = cs_chol (A,drop)
%CS_CHOL sparse Cholesky factorization.
%   L = cs_chol(A) is the same as L = chol(A)', using triu(A).
%   [L,p] = cs_chol(A) first orders A with p=cs_amd(A), so that L*L' = A(p,p).
%   A second optional input argument controls whether or not numerically zero
%   entries are removed from L.  cs_chol(A) and cs_chol(A,1) drop them;
%   cs_chol(A,0) keeps them.  They must be kept for cs_updown to work properly.
%
%   See also CS_AMD, CS_UPDOWN, CHOL, AMD, SYMAMD.

%   Copyright 2006, Timothy A. Davis.
%   http://www.cise.ufl.edu/research/sparse
error ('cs_chol mexFunction not found') ;
```

The `mex` and `mx` functions used by CSparse are listed below.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

`mex.h`	required mexFunction include file
`mexErrMsgTxt`	print error message and abort the mexFunction
`mexFunction`	required name of a mexFunction
`mexMakeMemoryPersistent`	ensures memory persists after mexFunction completes
`mxArray`	a MATLAB array
`mxCalloc`	MATLAB equivalent of the C `calloc` function
`mxCreateDoubleMatrix`	create a MATLAB full matrix
`mxCreateDoubleScalar`	create a MATLAB scalar (a 1-by-1 matrix)
`mxCreateSparse`	create a MATLAB sparse matrix
`mxFree`	MATLAB equivalent of the C `free` function
`mxGetJc`	get the column pointers of a MATLAB sparse matrix
`mxGetIr`	get the row indices of a MATLAB sparse matrix
`mxGetPr`	get the numerical values of a MATLAB sparse matrix
`mxGetM`	get number of rows of a MATLAB matrix
`mxGetN`	get number of columns of a MATLAB matrix
`mxGetNumberOfElements`	get number of entries of a MATLAB matrix
`mxGetNzmax`	get the maximum number of entries of a MATLAB matrix
`mxGetScalar`	get the value of a MATLAB scalar
`mxGetString`	get a MATLAB string
`mxIsChar`	true if a MATLAB matrix is a string
`mxIsComplex`	true if a MATLAB matrix is complex
`mxIsDouble`	true if a MATLAB matrix is double
`mxIsSparse`	true if a MATLAB matrix is sparse
`mxMalloc`	MATLAB equivalent of the C `malloc` function
`mxRealloc`	MATLAB equivalent of the C `realloc` function
`mxSetJc`	set the column pointers of a MATLAB sparse matrix
`mxSetIr`	set the row indices of a MATLAB sparse matrix
`mxSetPr`	set the numerical values of a MATLAB sparse matrix
`mxSetM`	set number of rows of a MATLAB matrix
`mxSetN`	set number of columns of a MATLAB matrix
`mxSetNzmax`	set the maximum number of entries of a MATLAB matrix

10.5 Further reading

Gilbert, Moler, and Schreiber introduced sparse matrices into MATLAB [105], including the first implementation of the sparse backslash. Additional sparse matrix functions of Amestoy, Davis, and Duff (AMD [1, 2]), Davis and Duff (UMFPACK [27, 28, 31, 32]), Davis, Gilbert, Larimore, and Ng (COLAMD [33, 34]), Davis, Hager, Chen, and Rajamanickam (CHOLMOD [30]), and Lehoucq, Sorensen, and Yang (ARPACK [144, 145, 188]) have been included. `condest` is based on Higham and Tisseur's [136] method, a generalization of Hager's 1-norm estimator [123]. Penny Anderson, Bobby Cheng, and Pat Quillen have written many of the sparse matrix methods in MATLAB. For more information on MATLAB, see Higham and Higham [133] or Davis and Sigmon [38]. Duff [47] discusses how random matrices (`sprand`, `sprandn`, and `sprandsym`) can give misleading results when factorized.

Exercises

10.1. Compare the performance (speed and accuracy) of the CSparse mexFunctions and MATLAB, using a range of large sparse matrices from real applications.